

Distributed Systems — Docker & Local Kubernetes Project

Technical Report — MSc DE1 — Rebecca Ouattara

Starter application	https://github.com/ubc/flask-sample-app
Repository	https://github.com/ORebecca/msc-de1-distributed-systems-docker-k8s
Docker Hub image	rebecca16ouatt/msc-de1-flask-app, tags 1.0.0 (deployed), latest, 1.1.0 (demo only)
Local cluster	kind, 1 control-plane + 2 workers, namespace msc-de1-project

1. Baseline

Before touching Docker at all, I ran the original app as-is to make sure it actually worked and to have something to compare the containerized version against later.

Steps: created a venv, installed requirements.txt (just flask==2.3.3), ran `python -m unittest discover tests` — the 4 existing tests all passed (`test_hello_route`, `test_add_item_route`, `test_get_item_route`, `test_get_nonexistent_item_route`) — then started the app with `python run.py` and hit every route with curl: `GET /` (200), `GET /items` (200), `POST /items` (201), `GET /items/<id>` (200, or 404 for an id that doesn't exist). Everything matched what the starter README said it should do. Raw output is in `evidence/baseline/` if you want to check it.

2. Dockerization

I wrote the Dockerfile myself, trying to hit every point on the assignment's checklist: an official Python base image, a proper `WORKDIR`, copying dependencies before the app source so Docker can cache that layer, no leftover caches, a non-root user, one exposed port, `exec-form CMD`, a `HEALTHCHECK`.

Why I switched base images

My first version used `python:3.12-slim` (Debian). It worked fine, but when I ran it through Trivy the results weren't great: **150 vulnerabilities, 44 of them HIGH**, almost all in Debian OS packages this app never even touches — things like `util-linux`, `login`, `perl-base`. None of them had a fix available yet either, so patching wasn't really an option. Switching the base to `python:3.12-alpine` got the OS-level findings to **zero** and dropped the image from **~198MB to ~78MB** in the process. I also removed `pip`, `setuptools` and `wheel` after installing the dependencies, since they're only needed at build time and Trivy was flagging vulnerabilities in those too.

Non-root

The image creates its own system user (appuser/appgroup, which end up as uid 100 / gid 101 on Alpine) and switches to it before the app starts. I checked this with `docker exec ... id`, which comes back as `uid=100(appuser) gid=101(appgroup)` — confirmed not root.

What I actually tested

`docker build`, `docker run`, hit every route from the host, checked `docker logs` for the request traffic, used `docker inspect` to confirm the non-root user and that only port 5000 is exposed, then a clean `stop/rm`. I did the same with `docker compose up`, and hardened that one a bit further for local testing: read-only root filesystem (with a `tmpfs` for `/tmp`), all capabilities dropped, and `no-new-privileges`. Nowhere in the setup is there a privileged container, a mounted Docker socket, or host networking.

3. Security

Vulnerability scan

Tool	Trivy 0.74.0
Image scanned	rebecca16ouatt/msc-de1-flask-app:1.0.0 (the Alpine version, final)
Findings by severity	CRITICAL: 0, HIGH: 0, MEDIUM: 0, LOW: 1
What changed after scanning	Debian → Alpine base (dropped 150 OS findings); removed pip/setuptools/wheel post-install (dropped their findings too)
What's left	CVE-2026-27205 (LOW): a Flask 2.3.3 issue around improper session-data caching leading to information disclosure. I didn't patch it because doing so would mean bumping Flask past the version the starter project pins in <code>requirements.txt</code> , and honestly it doesn't apply here anyway — this app never sets or reads Flask sessions or cookies, so that code path never runs.

Full scan output is saved as `security/vulnerability-scan.txt`.

SBOM

Generated with Syft 1.51.0, CycloneDX 1.7 JSON format (`security/sbom.cdx.json`) — 938 components covering every OS package and Python dependency actually present in the image.

Other things I locked down

- `allowPrivilegeEscalation: false` and all capabilities dropped in Kubernetes; same idea (`cap_drop: [ALL]`, `no-new-privileges`) in Compose.
- Read-only root filesystem in both Compose and Kubernetes — I checked that the app still starts, passes its health check, and serves every route fine with an `emptyDir/ tmpfs` for `/tmp`.

- CPU/memory requests and limits on the Deployment, so a misbehaving pod can't eat the whole cluster's resources.
- No credentials anywhere in the Dockerfile, image layers, or repo — the app doesn't need any.

4. Docker Hub

Repo: <https://hub.docker.com/r/rebecca16ouatt/msc-de1-flask-app>. I pushed 1.0.0 (the exact version the Kubernetes Deployment uses) and latest for the final image, plus a separate 1.1.0 built later just for the rolling-update demo in section 6.

To make sure publishing actually worked and wasn't just a local artifact, I removed the local image entirely, pulled it back from Docker Hub, ran it, and re-tested every route against that freshly-pulled copy. Same results as before.

5. Kubernetes

Cluster

Local kind cluster defined in `kind/kind-config.yaml`: 1 control-plane node, 2 worker nodes, all came up Ready.

What's deployed, in the `msc-de1-project` namespace

Object	What it does
Namespace	Keeps everything for this project grouped together.
ConfigMap	Holds the PORT value instead of hard-coding it anywhere.
Deployment	2 replicas of <code>rebecca16ouatt/msc-de1-flask-app:1.0.0</code> , container port 5000 declared explicitly, readiness and liveness probes hitting GET /, CPU/memory requests (50m/64Mi) and limits (250m/128Mi), a RollingUpdate strategy with <code>maxSurge: 1 / maxUnavailable: 0</code> , and a securityContext covering <code>runAsNonRoot</code> , the exact uid/gid the image uses (100/101), <code>allowPrivilegeEscalation: false</code> , dropped capabilities, <code>seccompProfile: RuntimeDefault</code> , and a read-only root filesystem (with an <code>emptyDir</code> for <code>/tmp</code>).
Service	ClusterIP, routes port 80 to the pods' port 5000.
NetworkPolicy	Only allows ingress to the app from inside the same namespace, and egress limited to DNS.

One thing worth flagging honestly: kind's default networking (kindnet) doesn't actually enforce NetworkPolicy. The policy documents what access should be allowed, but without a policy-aware CNI like Calico or Cilium installed on top, it's not actively blocking anything. I didn't have time to add that given the project deadline, so I'm noting it here instead.

6. Distributed behavior

Replication and service discovery

Both pods came up `Running/Ready`, scheduled on the two different worker nodes. `kubectl get endpoints` showed both pod IPs behind the Service, and I could reach the API through the Service via `kubectl port-forward`.

Self-healing

I deleted one running pod on purpose (`kubectl delete pod ...`) to see what would happen. Within a few seconds the Deployment had already created a replacement — nothing I had to trigger manually. That's the desired-state reconciliation loop at work: the controller keeps comparing what's actually running against what the spec says should be running, and fixes the difference on its own. A plain `docker run` container obviously doesn't do that; once it's dead, it's dead.

Scaling

`kubectl scale deployment msc-de1-flask-app --replicas=3` brought up a third pod without any other changes; the Deployment reported 3/3 shortly after. Scaling back down to 2 cleanly removed the extra one.

Rolling update and rollback

For this part I made a small, visible change — the `/` route now says "Hello, Flask! (v1.1.0)" — updated the matching test, rebuilt, and pushed the image as 1.1.0. Then I ran `kubectl set image deployment/msc-de1-flask-app msc-de1-flask-app=...:1.1.0`, watched `kubectl rollout status` replace the pods one at a time (the app stayed available throughout, since `maxUnavailable: 0`), and checked `kubectl rollout history` to see the new revision recorded. The Service started returning the new text right away. To finish the demo, `kubectl rollout undo` brought it back to the previous revision — both pods ended up back on 1.0.0, and the API's response reverted to the original text.

7. Conclusion

Doing this project meant dealing with the same small app in four different forms — a bare process, a Docker container, a Compose service, a Kubernetes workload — and trying to keep the security posture consistent across all of them instead of only hardening the parts that were easiest. The clearest lesson for me was just how much the base image choice matters: swapping Debian-slim for Alpine took the vulnerability count from 150 (44 HIGH) down to a single LOW finding, and that was a lot less effort than trying to patch individual OS packages one by one. Watching the self-healing and rolling-update demos actually run also made the theory click in a way reading about it hadn't — seeing a pod get killed and silently replaced, or watching the API stay reachable the entire time an update rolled out because of `maxUnavailable: 0`, made "desired state" feel like something concrete instead of a term from a slide.

If I were taking this toward an actual production deployment, the first thing I'd change is the web server: Flask's built-in dev server is still what's running inside the container, and it says outright it

isn't meant for production. I'd put something like Gunicorn behind it, and add a real Ingress with TLS instead of relying on `kubect1 port-forward` for access.